

# PartGuard - PartGuard Architecture Analysis

---

Given the description, PartGuard is not simply a CRUD application nor a pure AI application. It combines document processing, diagnostic reasoning, pricing analysis, multiple structured datasets, and privacy-preserving local inference. Consequently, a combination of architectural styles is more appropriate than a single style.

## Recommended Architectural Styles

### 1. Layered Architecture (Primary Style)

This should be the dominant architectural style.

#### Why it fits

The description explicitly separates responsibilities into:

- Receipt ingestion
- OCR processing
- Semantic extraction
- Diagnostic inference
- Pricing comparison
- Dataset coordination
- Visualization
- Persistence

These responsibilities naturally map to layers with controlled dependency relationships.

#### Possible Layer Structure

| Layer          | Responsibilities                                                         |
|----------------|--------------------------------------------------------------------------|
| Presentation   | Receipt capture, symptom entry, dashboards, visualizations               |
| Application    | Workflow orchestration and use-case coordination                         |
| Domain         | Repair analysis rules, pricing logic, diagnostic concepts                |
| AI Processing  | Symptom interpretation, component association, recommendation generation |
| Data           | MSRP datasets, OBD-II repositories, part catalogs, NHTSA datasets        |
| Infrastructure | OCR engine, storage, PDF handling, external integrations                 |

This is likely the most suitable overall architectural style.

## 2. Pipeline Architecture (Document Processing Subsystem)

The receipt-analysis workflow strongly resembles a processing pipeline.

### Pipeline Stages



### Why it fits

- Sequential transformations
- Clearly defined intermediate artifacts
- Easy stage replacement
- Supports independent testing

The receipt-processing subsystem is essentially a document-analysis pipeline embedded within a layered architecture.

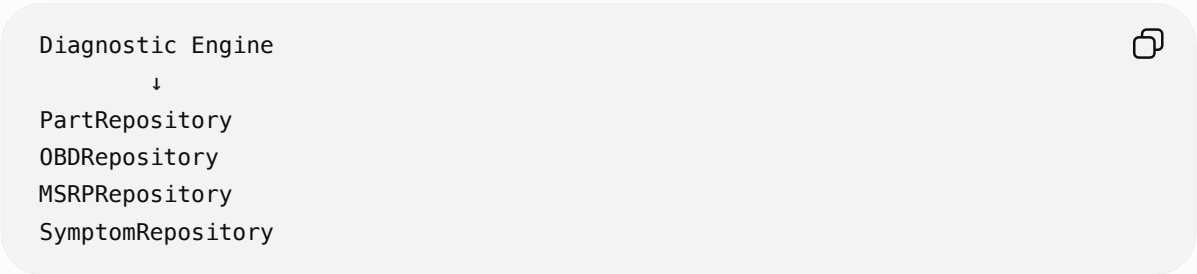
## 3. Repository Pattern (Dataset Access)

PartGuard relies heavily on structured automotive datasets.

Examples:

- MSRP repository
- Part catalog repository
- OBD-II repository
- Symptom association repository
- NHTSA-derived repository

A repository pattern provides:



## Benefits

- Dataset independence
- Easier dataset replacement
- Consistent access abstraction
- Reduced coupling to storage formats

---

## 4. Service-Oriented Domain Components

Within the Application and Domain layers, service-oriented decomposition is appropriate.

Examples:

- ReceiptAnalysisService
- OCRProcessingService
- PricingAnalysisService
- DiagnosticAnalysisService
- RecommendationService
- VehiclePartMatchingService

This provides clear responsibility boundaries while remaining lightweight compared to full microservices.

---

## 5. Ports-and-Adapters (Hexagonal) Pattern for External Dependencies

External technologies should be isolated behind adapters.

Examples:

- Apple Vision OCR adapter
- OBD-II adapter
- PDF adapter
- Local database adapter
- Marketplace synchronization adapter

Core Diagnostic Logic

↑

Ports

↑

Adapters

└ Vision OCR

└ OBD-II

└ SQLite/CoreData

└ Marketplace APIs



## Benefits

- Technology independence
- Easier testing
- Future OCR replacement

- Strong privacy boundary enforcement
- 

## Architectural Patterns Less Suitable

### Microservices

Not recommended.

Reasons:

- Primarily on-device execution
- Strong privacy requirements
- Local datasets
- No large-scale distributed deployment requirements

Microservices would add unnecessary complexity.

---

### Event-Driven Architecture

Only partially useful.

Events may help internally:

- ReceiptImported
- OCRCompleted
- DiagnosticCompleted

However, the system is fundamentally request-driven rather than event-driven.

---

### MVC

Useful only at the UI level.

MVC alone cannot adequately represent:

- OCR processing
- Diagnostic reasoning
- Dataset coordination
- Pricing analysis

It should be treated as a presentation-layer pattern rather than the overall system architecture.

---

## Recommended Combined Architecture

The strongest architectural characterization would be:

**A Layered Architecture incorporating a Document-Analysis Pipeline, Repository-Based Data Access, Service-Oriented Domain Components, and Hexagonal**

Adapters for external integrations.

Or more formally:

PartGuard adopts a layered architectural style in which automotive repair analysis responsibilities are separated across Presentation, Application, Domain, AI Processing, Data, and Infrastructure layers. Within this architecture, receipt and diagnostic processing are realized through a sequential pipeline pattern, structured automotive datasets are accessed through repository abstractions, business functionality is encapsulated within domain services, and external technologies such as OCR engines, OBD-II interfaces, and marketplace integrations are isolated through ports-and-adapters mechanisms. This combination supports privacy-preserving local execution, controlled dependency relationships, and clear separation of architectural responsibilities.



Based on the PartGuard description, the system can be decomposed into six architectural layers. The decomposition follows the separation of responsibilities described in the system specification and places processing, diagnostic reasoning, dataset management, and persistence concerns into distinct architectural units.

## Presentation Layer

Responsible for all user interaction, input acquisition, and result visualization.

### Composing Components

- Receipt Capture View
- PDF Import View
- Photo Selection View
- Symptom Entry View
- OBD-II Query View
- Repair Analysis Dashboard
- Pricing Comparison Dashboard
- Diagnostic Result View
- Recommendation Visualization View
- History and Search View
- Settings and Privacy View
- Visualization and Chart Components

### Responsibilities

- Collect repair receipts and diagnostic inputs

- Present pricing comparisons and diagnostic findings
  - Visualize recommendations and analysis results
  - Manage user navigation and interaction
- 

## **Application Layer**

Responsible for coordinating workflows and orchestrating interactions among lower layers.

### **Composing Components**

- Receipt Analysis Coordinator
- Diagnostic Analysis Coordinator
- Pricing Analysis Coordinator
- Recommendation Workflow Coordinator
- Receipt Processing Workflow Manager
- OBD-II Processing Coordinator
- Analysis Session Manager
- Result Aggregation Service
- History Management Service

### **Responsibilities**

- Orchestrate repair-analysis workflows
  - Coordinate OCR, extraction, pricing, and diagnostic operations
  - Aggregate intermediate processing results
  - Manage use-case execution
- 

## **Domain Layer**

Responsible for automotive repair concepts, business rules, and analytical logic.

### **Composing Components**

- Repair Record Model
- Automotive Part Model
- Vehicle Component Model
- Diagnostic Symptom Model
- OBD-II Code Model
- MSRP Comparison Model
- Pricing Analysis Engine
- Repair Verification Engine
- Diagnostic Reasoning Engine
- Recommendation Engine
- Automotive Knowledge Model
- Repair Cost Evaluation Rules

### **Responsibilities**

- Represent core automotive concepts
  - Evaluate repair pricing and verification rules
  - Perform repair-cost assessment
  - Maintain domain-specific reasoning logic
- 

## **AI Processing Layer**

Responsible for semantic interpretation and AI-assisted analysis.

### **Composing Components**

- Symptom Interpretation Engine
- Component Association Engine
- Diagnostic Inference Engine
- Recommendation Generation Engine
- Semantic Context Analyzer
- Fault Correlation Engine
- AI Result Ranking Engine

### **Responsibilities**

- Interpret symptom descriptions
  - Associate symptoms with automotive components
  - Infer potential diagnostic explanations
  - Generate structured recommendations
  - Rank and prioritize diagnostic findings
- 

## **Data Layer**

Responsible for structured automotive knowledge and dataset management.

### **Composing Components**

- MSRP Repository
- Automotive Part Catalog Repository
- OBD-II Repository
- Symptom-to-Component Repository
- NHTSA Diagnostic Dataset Repository
- Vehicle Component Repository
- Pricing Reference Repository
- Dataset Synchronization Manager
- Repository Query Services
- Dataset Index Manager

### **Responsibilities**

- Manage structured automotive datasets
- Provide indexed lookup services
- Support pricing and diagnostic analysis

- Coordinate dataset access

---

## Infrastructure Layer

Responsible for external technologies, storage, and platform-specific services.

### Composing Components

- Apple Vision OCR Adapter
- OCR Processing Engine
- Token Reconstruction Engine
- Receipt Storage Manager
- Local Database Manager
- PDF Processing Adapter
- Image Processing Adapter
- OBD-II Interface Adapter
- Marketplace Synchronization Adapter
- External Pricing Reference Adapter
- File Management Service
- Caching Service

### Responsibilities

- Execute OCR extraction
- Handle file and image processing
- Manage local persistence
- Access external data sources when required
- Provide platform-specific capabilities

---

## Resulting Layered Architecture

Presentation

↓

Application

↓

Domain

↓

AI Processing

↓

Data

↓

Infrastructure



This decomposition reflects the major architectural responsibilities described for PartGuard: receipt ingestion, OCR processing, semantic extraction, diagnostic inference, pricing comparison, dataset coordination, visualization management, and persistence handling, while maintaining explicit separation between user interaction, workflow orchestration, automotive reasoning, AI-assisted analysis, dataset management, and infrastructure concerns.

Establish allowed and forbidden dependencies among components across layers. &  
Dependency definition and architectural constraint specification

For PartGuard, the dependency constraints should enforce the architectural intent described in the system specification: privacy-preserving local processing, separation of responsibilities, controlled data access, and isolation of AI-related functionality. The following dependency rules define which inter-layer dependencies are permitted and prohibited.

## Allowed Dependencies

### Presentation Layer

The Presentation layer may depend on:

- Application Layer
- Domain Layer (read-only domain models and view representations)

#### Examples

- ReceiptCaptureView → ReceiptAnalysisCoordinator
  - DiagnosticDashboard → DiagnosticResultModel
  - PricingComparisonView → MSRPComparisonModel
- 

### Application Layer

The Application layer may depend on:

- Domain Layer
- AI Processing Layer
- Data Layer

#### Examples

- ReceiptAnalysisCoordinator → RepairVerificationEngine
  - DiagnosticAnalysisCoordinator → DiagnosticInferenceEngine
  - PricingAnalysisCoordinator → MSRPRepository
- 

### Domain Layer

The Domain layer may depend on:

- Data Layer

#### Examples

- PricingAnalysisEngine → MSRPRepository

- DiagnosticReasoningEngine → SymptomRepository
- RecommendationEngine → OBDRepository

The Domain layer should remain independent of Presentation and Infrastructure concerns.

---

## AI Processing Layer

The AI Processing layer may depend on:

- Domain Layer
- Data Layer

### Examples

- SymptomInterpretationEngine → DiagnosticSymptomModel
- ComponentAssociationEngine → SymptomRepository
- DiagnosticInferenceEngine → OBDRepository

AI processing may consume domain concepts and structured datasets but should not directly access infrastructure services.

---

## Data Layer

The Data layer may depend on:

- Infrastructure Layer

### Examples

- MSRPRepository → LocalDatabaseManager
  - OBDRepository → FileManagementService
  - DatasetSynchronizationManager → MarketplaceSynchronizationAdapter
- 

## Infrastructure Layer

The Infrastructure layer should not depend on higher layers.

Allowed dependencies:

- Internal infrastructure dependencies only

Examples:

- OCRProcessingEngine → AppleVisionOCRAdapter
  - LocalDatabaseManager → FileManagementService
- 

## Forbidden Dependencies

### Presentation Layer

Forbidden:

- Presentation → AI Processing
- Presentation → Data
- Presentation → Infrastructure

## Invalid Examples

DiagnosticView  
→ MSRPRepository



ReceiptCaptureView  
→ AppleVisionOCRAAdapter



All such interactions must pass through Application services.

---

## Application Layer

Forbidden:

- Application → Infrastructure

### Invalid Example

ReceiptAnalysisCoordinator  
→ AppleVisionOCRAAdapter



Infrastructure access must be mediated through Data-layer abstractions or dedicated services.

---

## Domain Layer

Forbidden:

- Domain → Presentation
- Domain → Application
- Domain → Infrastructure

### Invalid Examples

RepairVerificationEngine  
→ ReceiptCaptureView



DiagnosticReasoningEngine  
→ OCRProcessingEngine



The Domain layer must remain technology-independent.

---

## AI Processing Layer

Forbidden:

- AI Processing → Presentation
- AI Processing → Application
- AI Processing → Infrastructure

## Invalid Examples

DiagnosticInferenceEngine  
→ ReceiptCaptureView



ComponentAssociationEngine  
→ AppleVisionOCRAAdapter



AI components should operate on structured domain information and repositories rather than platform technologies.

---

## Data Layer

Forbidden:

- Data → Presentation
- Data → Application
- Data → Domain
- Data → AI Processing

Repositories should not invoke business logic or workflow orchestration.

## Invalid Example

MSRPRepository  
→ PricingAnalysisEngine



## Infrastructure Layer

Forbidden:

- Infrastructure → Presentation
- Infrastructure → Application
- Infrastructure → Domain
- Infrastructure → AI Processing
- Infrastructure → Data

Infrastructure components provide services but do not coordinate business logic.

## Invalid Example

AppleVisionOCRAAdapter  
→ DiagnosticInferenceEngine



# Layer Dependency Matrix

| From           | Presentation | Application | Domain | AI Processing | Data | Infrastructure |
|----------------|--------------|-------------|--------|---------------|------|----------------|
| Presentation   | ✓            | ✓           | ✓      | ×             | ×    | ×              |
| Application    | ×            | ✓           | ✓      | ✓             | ✓    | ×              |
| Domain         | ×            | ×           | ✓      | ×             | ✓    | ×              |
| AI Processing  | ×            | ×           | ✓      | ✓             | ✓    | ×              |
| Data           | ×            | ×           | ×      | ×             | ✓    | ✓              |
| Infrastructure | ×            | ×           | ×      | ×             | ×    | ✓              |

## Architectural Constraint Specification

The architecture follows a **strict layered dependency model**:

1. User-interface concerns are isolated in the Presentation layer.
2. Workflow coordination is centralized in the Application layer.
3. Automotive repair reasoning and pricing logic reside in the Domain layer.
4. AI-assisted interpretation and recommendation functionality reside in the AI Processing layer.
5. Automotive datasets and repositories are encapsulated within the Data layer.
6. OCR, storage, file handling, OBD-II access, and external integrations are confined to the Infrastructure layer.
7. Dependencies may only follow the allowed relationships defined above; any direct dependency violating these rules constitutes an architectural violation. This ensures preservation of responsibility boundaries, controlled information flow, technology independence of business logic, and privacy-preserving local processing.



Refine the internal structure of each layer using component-level architectural representations.

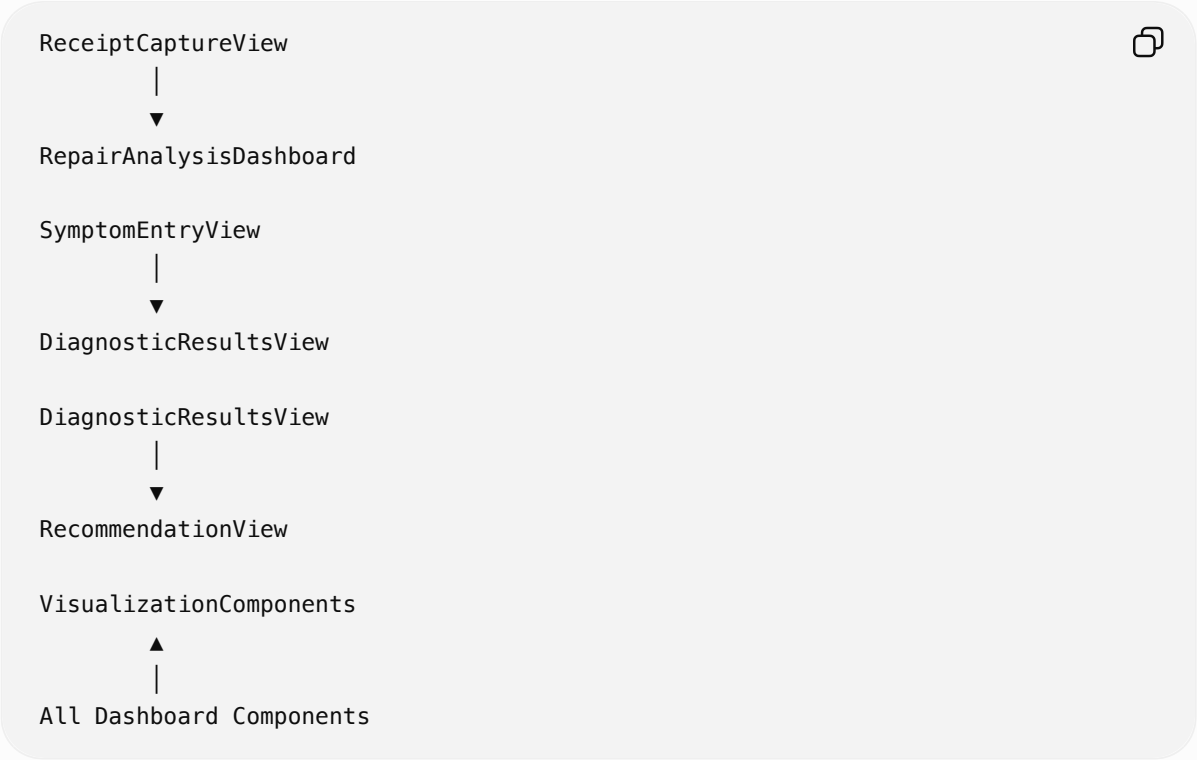
A useful refinement is to move from **layer-level architecture** to **component-level architecture**, where each layer is decomposed into collaborating components with explicit internal relationships. The resulting structure provides a more implementable architecture while preserving the previously defined dependency constraints.

## Presentation Layer

Responsible for user interaction and visualization.

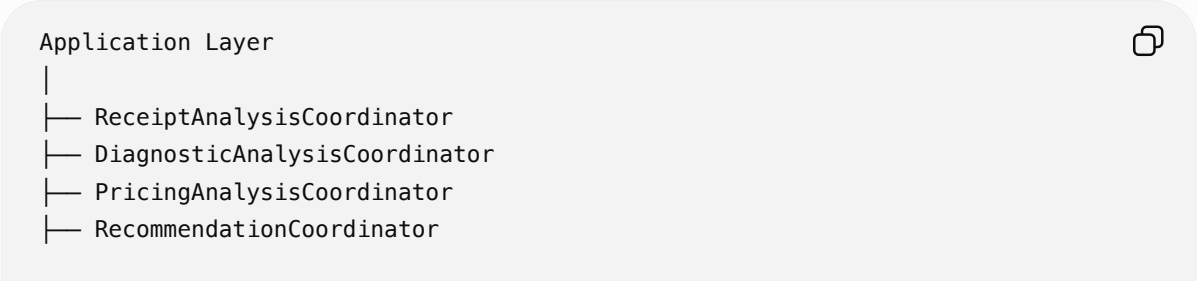


Internal Relationships



Application Layer

Responsible for use-case orchestration and workflow coordination.



```
|
├─ ReceiptProcessingWorkflow
├─ DiagnosticWorkflow
├─ PricingWorkflow
|
├─ ResultAggregationService
├─ AnalysisSessionManager
|
└─ HistoryManagementService
```

## Internal Relationships

ReceiptAnalysisCoordinator



ReceiptProcessingWorkflow

DiagnosticAnalysisCoordinator



DiagnosticWorkflow

PricingAnalysisCoordinator



PricingWorkflow

ReceiptProcessingWorkflow

DiagnosticWorkflow

PricingWorkflow



ResultAggregationService

ResultAggregationService



AnalysisSessionManager



## Domain Layer

Responsible for automotive concepts and business rules.

Domain Layer



Entities



├─ RepairRecord



├─ AutomotivePart



├─ VehicleComponent



├─ DiagnosticSymptom



├─ OBDCode



└─ MSRPComparison

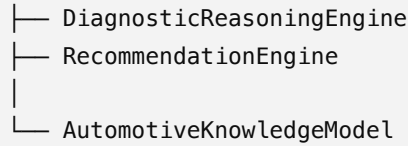


PricingAnalysisEngine

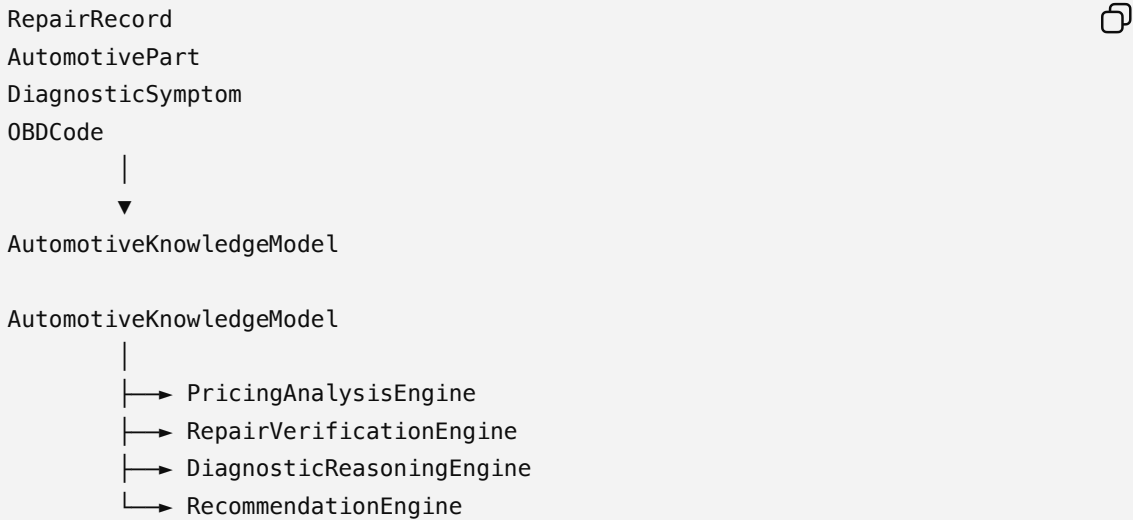


RepairVerificationEngine



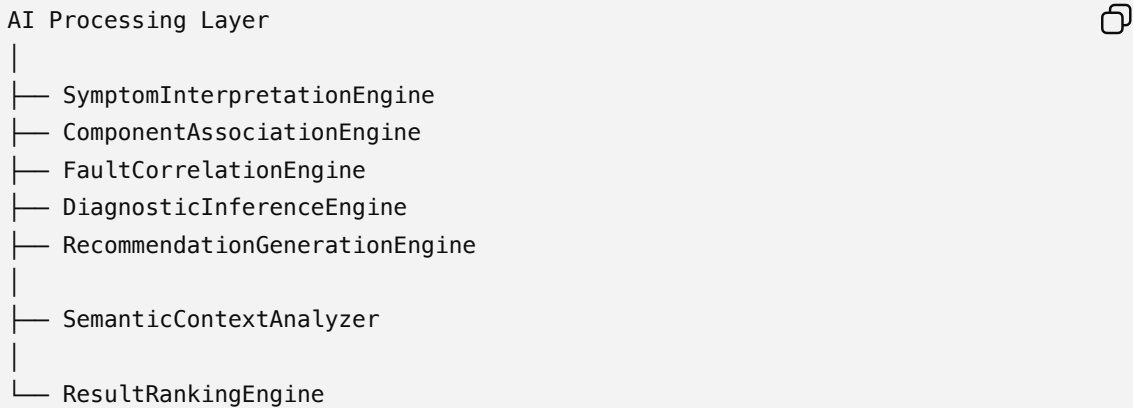


### Internal Relationships

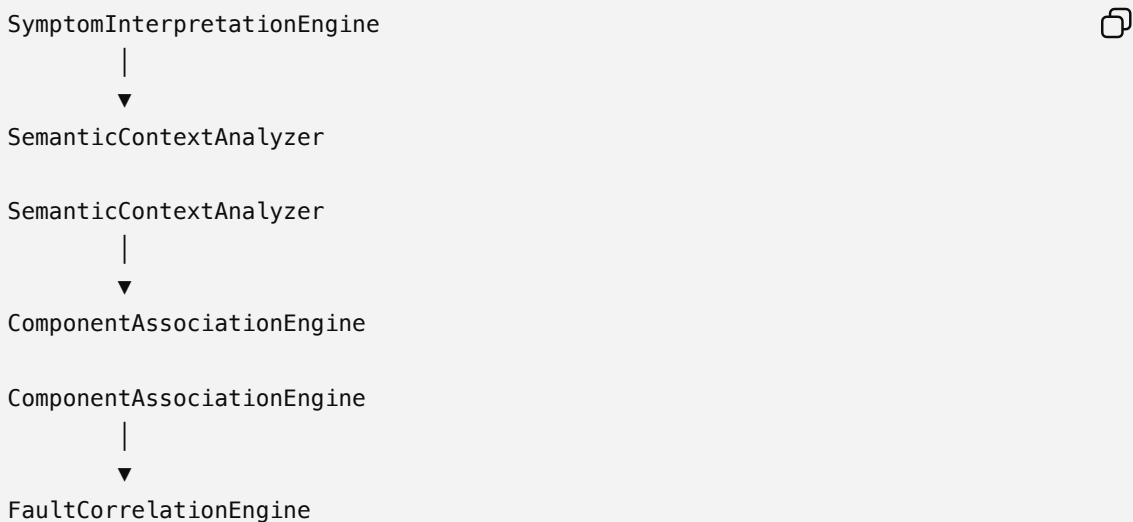


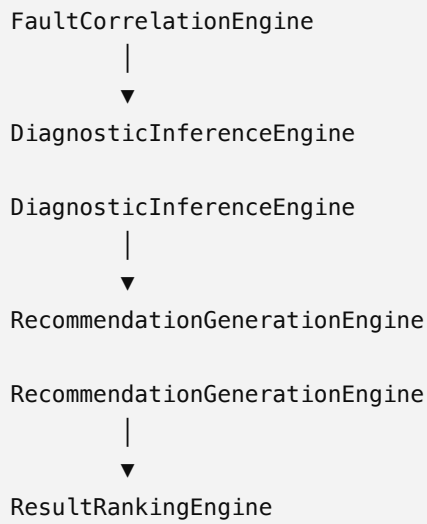
## AI Processing Layer

Responsible for AI-assisted interpretation and inference.



### Internal Relationships

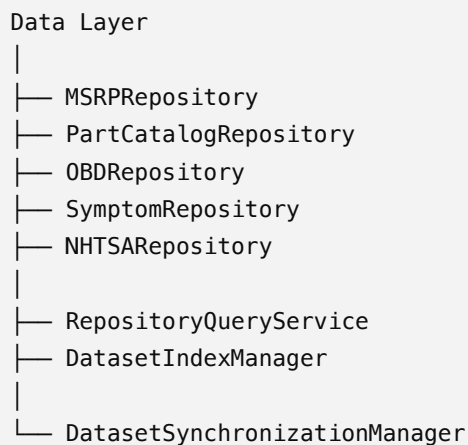




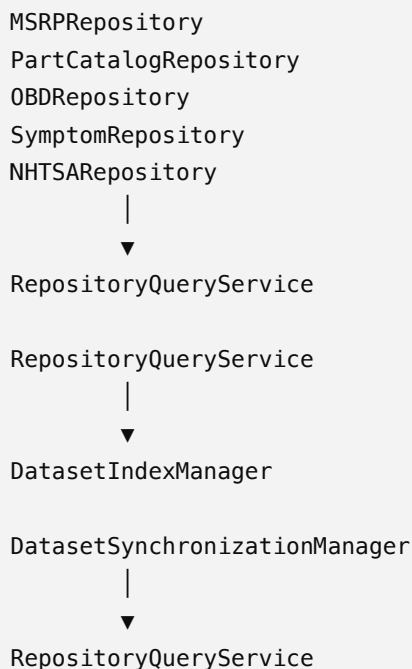
This creates a structured diagnostic inference pipeline within the AI layer.

## Data Layer

Responsible for dataset access and repository management.

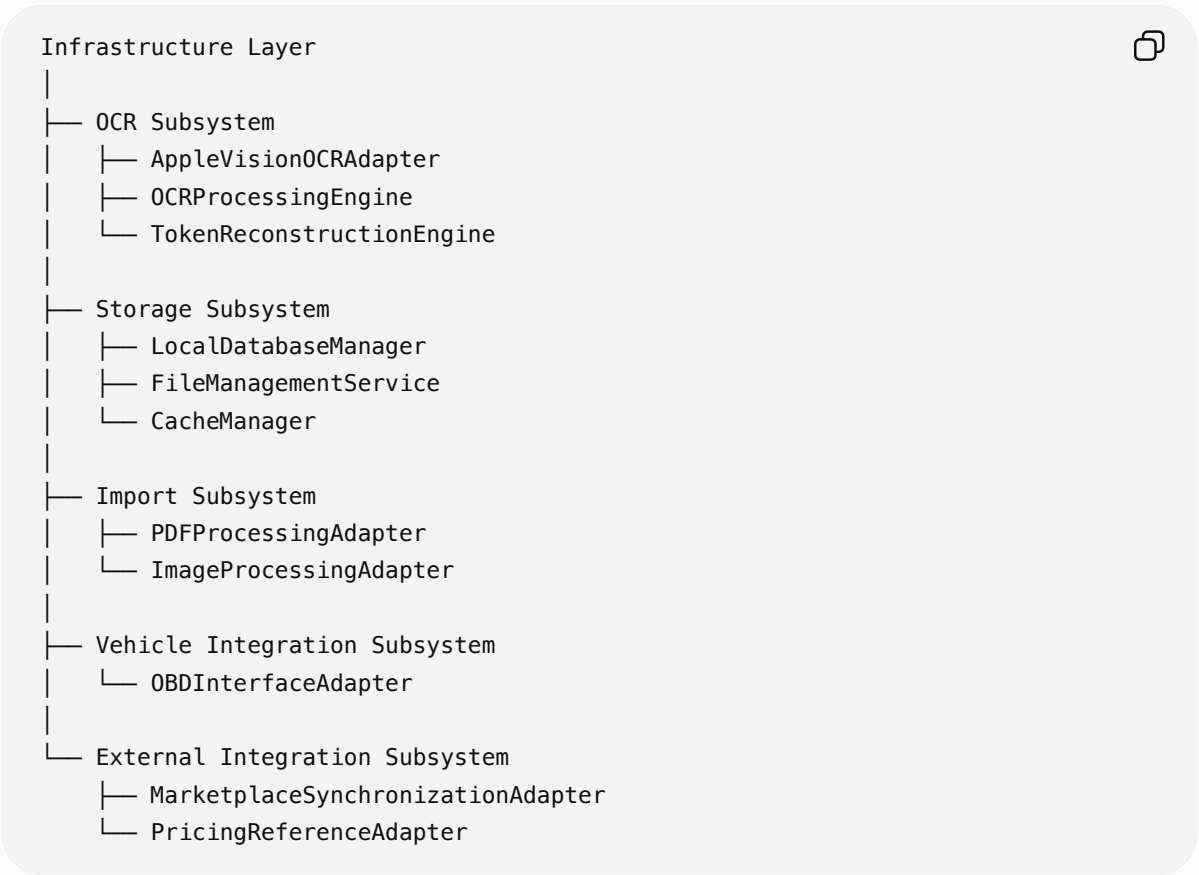


## Internal Relationships

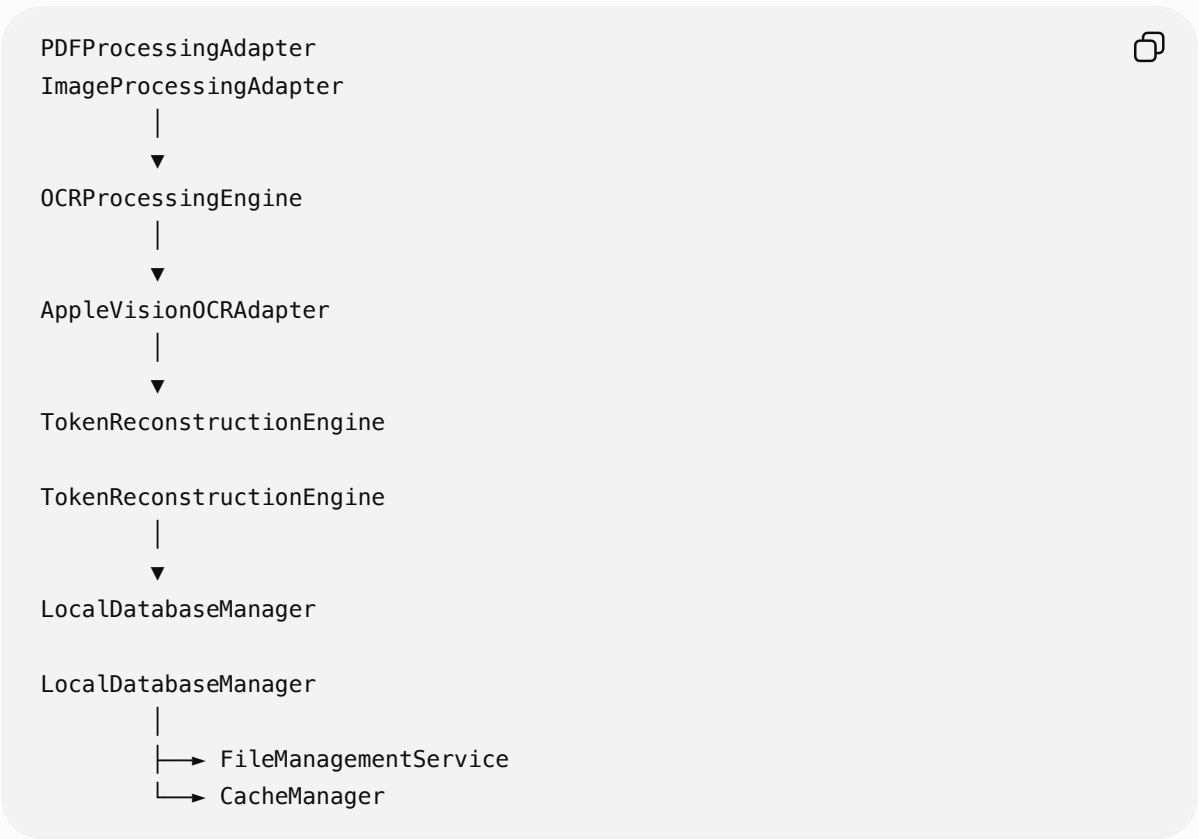


# Infrastructure Layer

Responsible for technology-specific implementation.

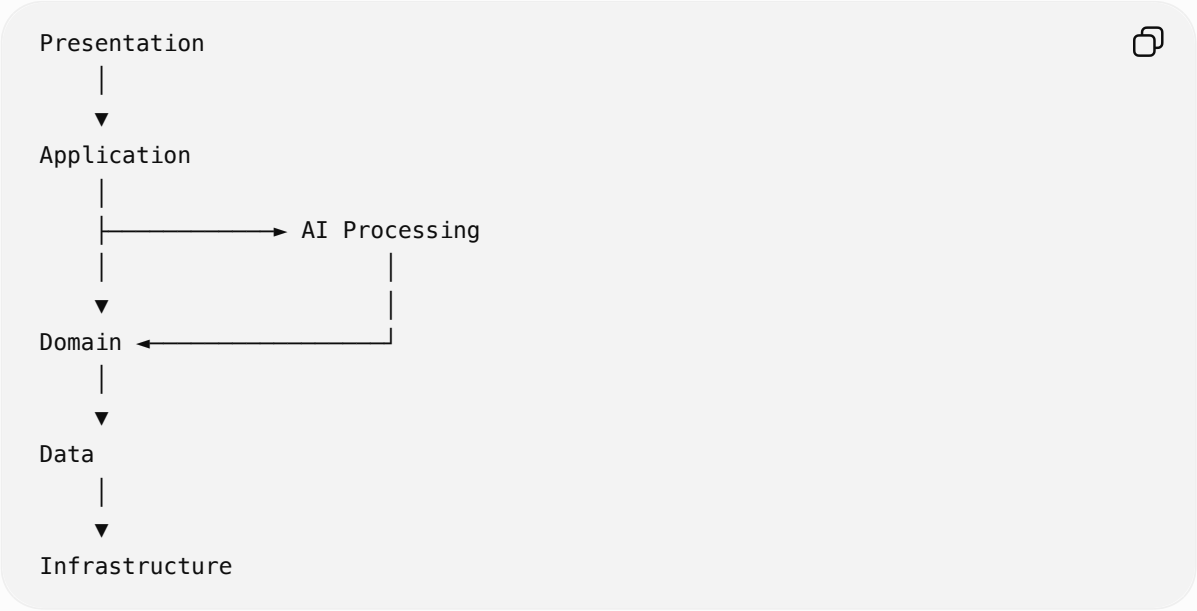


## Internal Relationships

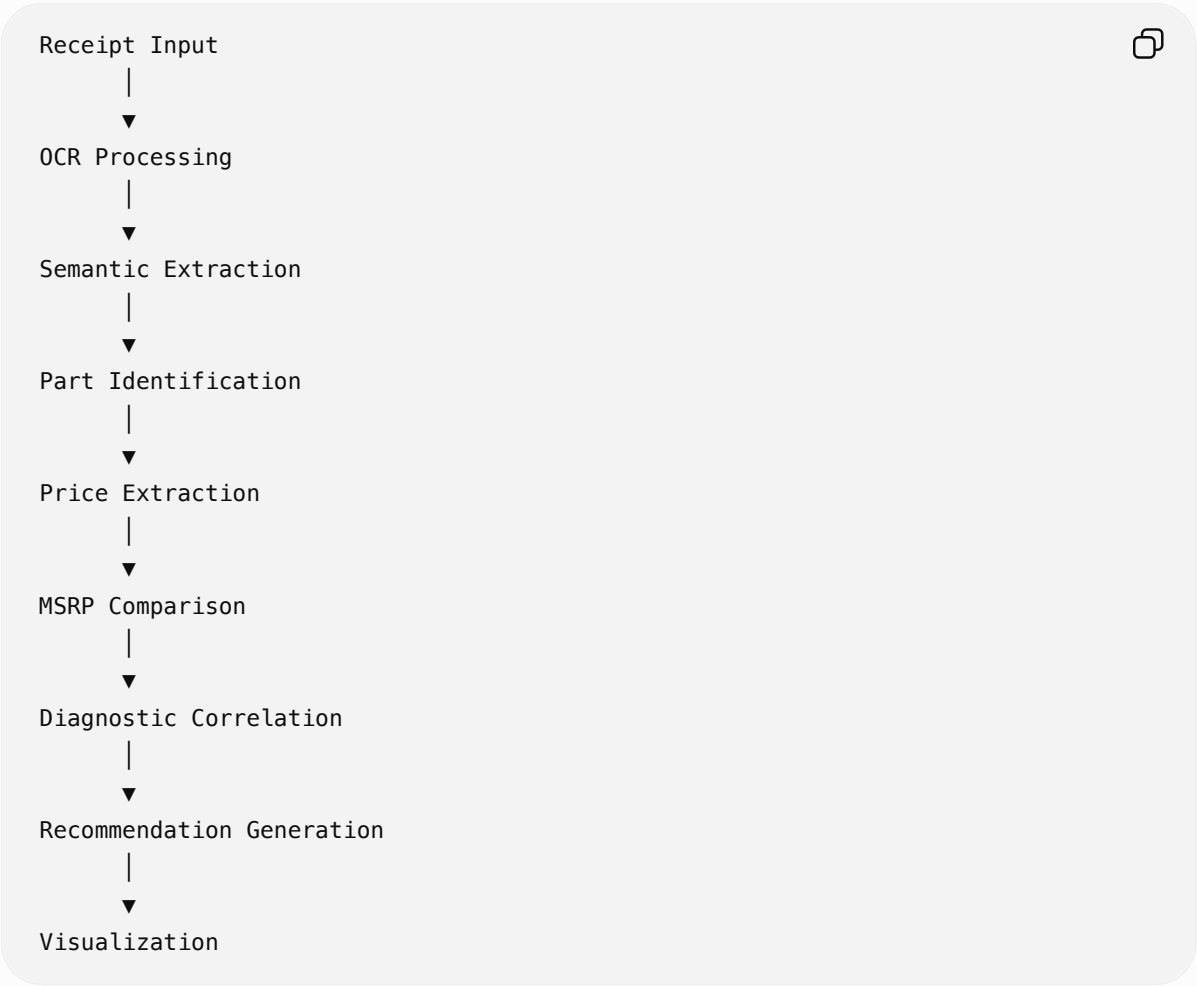


# Integrated Component Architecture

The overall component architecture can be summarized as:



with the principal processing path:



This refinement yields a component-level architecture suitable for subsequent architectural views, UML component diagrams, DILAS-style layer allocation, and implementation planning.